Explainability-Guided Distillation of Robust UAV Navigation Policies under Stochastic Dynamics

Anonymous Submission

Affiliation withheld for review

Abstract. Reinforcement learning is increasingly attractive for control, but learned policies remain difficult to trust because they are black-box mappings, sensitive to hyperparameters, and hard to tune after deployment. We study whether a successful UAV-navigation policy can also be recovered as a compact physical control law. To do so, we build a custom Gymnasium–PyBullet environment with noisy sensing, Kalman-filtered state estimation, aerodynamic drag, wind gusts, and obstacle drift, and train SAC, TD3, DDPG, and PPO under the same stochastic dynamics. We then analyze the learned controllers with SHAP, LIME, decision-tree surrogates, vector-field visualizations, and rollout-aware equation distillation. The main result is that the strongest policy can be expressed as a compact damped contextual potential field with clear goal-attraction, velocity-damping, and local-repulsion terms. This positions policy distillability as a practical evaluation axis alongside reward and success rate, with clear implications for inspection, manual retuning, and eventual sim-to-real transfer.

Keywords: UAV navigation · reinforcement learning · Kalman filtering · explainable AI · policy distillation

1 Introduction

Learning-based UAV navigation is attractive because hand-crafted obstacle-avoidance rules often become brittle once sensing becomes noisy, obstacles drift, or dynamics deviate from the nominal model. At the same time, high-performing deep RL policies are difficult to audit, difficult to transfer, and difficult to compare beyond scalar reward. This raises a broader question: if a controller succeeds in a noisy environment, can its behavior be explained as a compact control law rather than only as a black-box network?

We address that question in a controlled simulation study. The environment is a custom Gymnasium-style task [3] backed by PyBullet [4], with a quadrotor-like agent moving in the horizontal plane of a 3D scene while navigating through mixed-shape obstacles toward a fixed goal. All experiments reported in the paper use the same stochastic setting: trajectory penalties, measurement noise, a Kalman filter [8], aerodynamic drag, wind gusts, and obstacle drift. These perturbations are not arbitrary. They target the main failure modes seen in early policies: poor state estimation, overconfident straight-line motion, and sensitivity

to mild distribution shift. In that sense, the environment behaves like a structured robustness benchmark inspired by sim-to-real randomization work [20,16], but with perturbations that remain interpretable.

The second part of the paper focuses on explanation and distillation. We combine decision-tree surrogates, LIME [17], SHAP [13], vector-field plots, Q-value heatmaps, and rollout-aware equation extraction. The main empirical finding is that the best SAC controller behaves like a damped contextual potential field [10]: action depends primarily on raw goal displacement, velocity damping, and a short-range obstacle repulsion term. This structure was not hard-coded. It emerged from training and was recovered post hoc. By contrast, TD3 and DDPG can achieve strong task success while remaining much harder to summarize with the same interpretable controller family.

The contribution is therefore a combined environment-design, robustness, and explainability study with three concrete outcomes:

1. a single-shot noisy PyBullet environment for UAV navigation with a Kalman-filter state-estimation layer;
2. an explainability pipeline that goes beyond feature attribution and validates distilled equations in closed-loop rollouts;
3. empirical evidence that policy success and policy distillability are different properties, with SAC emerging as the most faithful analytical surrogate in our setting.

The broader motivation is interpretability-by-design: rather than adding explanation as an afterthought, we treat closed-loop policy fidelity as the primary validation criterion for any distilled surrogate. This changes the evaluation question from "can the explanation approximate the policy locally?" to "can the explanation *replace* the policy in a full rollout?". The remainder of the paper is structured as follows. Section 2 surveys related work. Section 3 introduces the preliminaries, including the problem statement and controller setup. Section 4 describes the environment and training protocol. Section 5 presents the result analysis. Section 6 presents the explainability and distillation pipeline. Section 7 discusses limitations and conclusions.

2 Related Work

Deep RL has been used widely for navigation and obstacle avoidance in both ground and aerial robots, including continuous-control drone settings [14,7]. However, much of that literature focuses on success rate and transfer robustness rather than on whether the learned policy can be turned back into a compact control law. Benchmarking work such as Arena-Bench [9] has also emphasized how sensitive obstacle-avoidance performance can be to environment dynamics and evaluation protocol. Our environment design follows that line of thought: we do not assume that clean-navigation success is enough to demonstrate robustness.

There is also a clear line of work on interpretable or explainable RL. Surveys by Milani et al. [15] and related explainable-robotics work [11] show that explanation methods in RL range from saliency and attribution to symbolic surrogates, hierarchical skills, and policy extraction. SHAP [13] and LIME [17] have become standard post hoc explanation tools, while policy distillation [18] and verification-oriented extraction methods such as VIPER [1] aim to replace black-box policies with simpler surrogates. More recent work on concept-based policy distillation [2] further argues that interpretable latent concepts can preserve useful control structure.

Our work differs from those directions in two ways. First, the target is a continuous UAV-navigation controller, not a discrete toy domain or a concept-level explanation alone. Second, we treat *closed-loop fidelity* as the key evaluation criterion for distillation. A surrogate with high one-step R^2 but poor rollout behavior is not considered faithful enough. This is the central methodological difference from many explanation pipelines, which stop at importance rankings or local linear explanations.

Finally, combining filtering and RL for navigation is itself not new. Marino and Guglieri [14], for example, combine an interacting multiple model Kalman filter with PPO for drone navigation. Our contribution is therefore narrower and, we believe, more defensible: in a custom PyBullet UAV environment, we combine a structured noise-and-dynamics benchmark, a Kalman-filter ablation, and rollout-validated analytical distillation. The novelty lies less in any single ingredient than in the way they are tied together to show a measurable gap between policy success and policy distillability.

3 Preliminaries

3.1 Problem Statement

We formulate the UAV-navigation task as a continuous partially observable Markov decision process (POMDP), defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$. At each time step t, the environment occupies a latent state $s_t \in \mathcal{S}$, the agent receives an observation $o_t \in \mathcal{O}$, selects an action $a_t \in \mathcal{A}$ according to a policy $\pi(a_t \mid o_t)$, and receives a scalar reward $r_t = \mathcal{R}(s_t, a_t)$. The environment then transitions to s_{t+1} according to $\mathcal{P}(s_{t+1} \mid s_t, a_t)$. The objective is to maximize the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}, \tag{1}$$

where $\gamma \in [0, 1)$ controls the effective planning horizon. For the underlying latent-control problem, the optimal action-value function satisfies the Bellman optimality relation

$$Q^*(s, a) = \mathbb{E}\Big[r + \gamma \max_{a'} Q^*(s', a') \,\Big|\, s, a\Big]. \tag{2}$$

Our specific instantiation is a UAV-navigation problem embedded in a 3D PyBullet world, but with commanded motion intentionally constrained to the horizontal (x, y) plane. Altitude and out-of-plane attitude are fixed, so the agent solves a planar avoidance task while still interacting with 3D geometry for collision checking and rendering.

Latent state space $\mathcal{S}$ and observation space $\mathcal{O}$. The latent simulator state contains the true planar UAV kinematics, the fixed goal location, and the current obstacle field:

$$s_t = \left[x_t, y_t, v_{x,t}, v_{y,t}, g_x, g_y, \{o_{i,x}, o_{i,y}, \rho_i, \kappa_i\}_{i=1}^{8}\right], \tag{3}$$

where (x_t, y_t) and $(v_{x,t}, v_{y,t})$ are the UAV position and velocity, (g_x, g_y) is the goal location, and each obstacle contributes its planar position, effective radius ρ_i, and shape code $\kappa_i \in \{\texttt{sphere}, \texttt{box}, \texttt{cylinder}\}$.

The policy does not receive s_t directly. Instead, it acts on a 39-dimensional observation vector,

$$o_t = \left[\hat{x}_t, \hat{y}_t, \hat{v}_{x,t}, \hat{v}_{y,t}, \Delta g_x, \Delta g_y, d_g, \{o_{i,x}^{\text{obs}}, o_{i,y}^{\text{obs}}, \rho_i, \kappa_i\}_{i=1}^{8}\right] \in \mathbb{R}^{39}, \tag{4}$$

where $(\hat{x}_t, \hat{y}_t, \hat{v}_{x,t}, \hat{v}_{y,t})$ are Kalman-filtered position and velocity estimates, $(\Delta g_x, \Delta g_y)$ is the relative goal displacement, and d_g is the Euclidean distance to the goal. Obstacle coordinates in the observation are noisy, while obstacle radii and shape codes remain deterministic descriptors. This separation between s_t and o_t is precisely why the problem is treated as a POMDP rather than a fully observed MDP.

Action space $\mathcal{A}$. The action is a bounded planar control command

$$a_t = (a_{x,t}, a_{y,t}) \in [-1, 1]^2, \tag{5}$$

which is mapped inside the simulator to a commanded horizontal UAV velocity with maximum magnitude 15.000 m/s. The action therefore represents a continuous desired motion direction and speed in the plane, not a discrete steering primitive.

Reward function $\mathcal{R}$. The reward is a structured navigation objective designed to balance target reaching, safety, and trajectory quality:

$$\begin{aligned} r_t = {} & r_{\text{attr}} + r_{\text{align}} + r_{\text{repel}} + r_{\text{clear}} + r_{\text{goal}} \\ & - c_{\text{time}} - c_{\text{stall}} - c_{\text{smooth}} - c_{\text{near}} - c_{\text{safety}} - c_{\text{collision}}. \end{aligned} \tag{6}$$

Rather than present this as a single opaque scalar, we expose the behavioral components used in the environment:

- **Attraction/progress:** r_{attr} rewards reduction in goal distance with gain 12.0, plus an approach bonus $5\exp(-d_g/0.8)$ when the UAV is moving closer to the goal. The attractive contribution is clipped to $[-8, 8]$ to prevent uncontrolled reward growth.

- **Directional alignment:** $r_{\text{align}} = 0.75 \cos \theta_t$ rewards velocity aligned with the goal direction.
- **Obstacle interaction:** r_{repel} is a clipped negative repulsion term based on safety-clearance breach, while $c_{\text{safety}} = 2.5$ and $c_{\text{collision}} = 20.0$ impose additional penalties for entering the safety margin or colliding.
- **Trajectory quality:** r_{clear} rewards maintaining clearance from nearby obstacles, c_{smooth} penalizes jerk, c_{near} penalizes near misses, and $c_{\text{stall}} = 0.35$ penalizes low-progress low-speed stagnation far from the goal.
- **Time and success:** a per-step time cost $c_{\text{time}} = 0.30$ discourages dithering, while $r_{\text{goal}} = 35.0$ is awarded on successful goal reach.

An episode is counted as successful when the UAV reaches the goal region without collision, either by entering a 0.650 m goal radius or by remaining within 1.000 m of the goal for 120 consecutive simulator steps.

Transition dynamics $\mathcal{P}$. The transition kernel is governed by PyBullet. The environment steps at 240.000 Hz for up to 18.000 s per episode. In the noisy setting used throughout the paper, transitions include aerodynamic drag, wind gust disturbances, and small bounded obstacle drift around nominal anchor locations. From the policy's perspective, the combination of noisy observations and perturbed simulator dynamics makes the navigation problem explicitly stochastic. The Kalman filter therefore functions only as a state-estimation layer in the observation pipeline; it does not alter the underlying transition dynamics or act as a separate planner.

3.2 Reinforcement Learning Algorithms

We benchmark the environment using established continuous-control baselines: PPO [19] as the on-policy reference, and DDPG [12], TD3 [5], and SAC [6] as off-policy alternatives. Figure 1 summarizes the instantiated policy structures used in our study. The figure deliberately separates the actor backbone, the policy head, and the critic layout, because those distinctions matter more here than simply counting layers.

Despite their different training objectives, the backbones are not radically different. SAC and TD3 both use a 256-256-256 actor MLP in our runs, DDPG uses a shallower 256-256 actor, and PPO uses a 512-256-256 actor/value pair. The main structural differences are therefore in the policy head and critic arrangement: PPO uses separate actor and value streams updated from fresh rollouts only, DDPG uses a deterministic actor with a single critic, TD3 keeps a deterministic actor but adds twin critics and delayed policy updates, and SAC combines twin critics with a stochastic Gaussian actor and entropy regularization.

This architectural view is useful when reading the distillation results later in the paper. Since SAC and TD3 share the same actor depth in our implementation, SAC's better analytical surrogate cannot be explained as a trivial network-size effect. The more plausible explanation is that the stochastic SAC head and its regularized training objective yield a smoother policy surface, whereas TD3

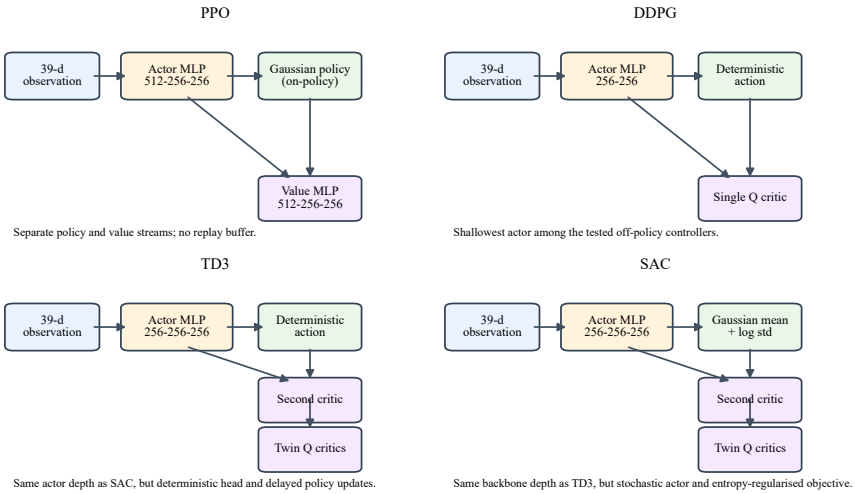

Fig. 1. Block-diagram summary of the policy-network structures used in the experiments. Widths are taken from the trained runs reported in Table 2. SAC and TD3 share the same actor depth; the key structural difference is that SAC uses a stochastic Gaussian actor with entropy regularization, whereas TD3 and DDPG use deterministic policy heads.

Table 1. Quantitative comparison of executable surrogate controllers for the main SAC policy in the shared noisy evaluation setting.

Method	R_x^2	R_y^2	Success	Trace dev. (m)	Endpoint dev. (m)
Sparse linear law	0.676	0.495	1/4	4.642	4.080
Depth-15 decision tree	0.993	0.991	4/4	1.240	0.330
SHAP compact equation	0.704	0.544	4/4	2.154	0.795

and DDPG expose sharper deterministic corrections that are harder to compress into a compact closed-form controller.

3.3 Explainability and Policy Distillation

To analyze the learned policies, we employ SHAP [13] for global feature attribution, LIME [17] for local directional explanations, and depth-limited decision trees [1] as rule-based surrogates. Crucially, we validate these explanations not merely by one-step R^2 fidelity, but by closed-loop rollout success on fixed evaluation seeds.

Table 1 now makes the ranking explicit. The saved depth-15 decision tree is the most faithful policy proxy overall: it achieves near-perfect one-step fit and preserves all four evaluation successes with the smallest trajectory error. The SHAP-guided compact law remains the best *analytical equation*: it is much more compact than the tree while still preserving all four evaluation successes.

4 Environment and Training Setup

4.1 Custom Gymnasium/PyBullet Environment

The environment is implemented with a Gymnasium interface [3] and simulated in PyBullet [4]. Although the simulator itself is fully 3D, the UAV motion is constrained to the horizontal plane so that the task remains a planar navigation problem with 3D collision geometry and rendering. The UAV starts near one of four corners of a 10.000 m square workspace and must reach a central goal while avoiding eight mixed-shape obstacles (spheres, cylinders, and boxes).

Episodes run at 240.000 Hz for up to 18.000 s. Success occurs when the UAV reaches within 0.650 m of the goal, or remains within 1.000 m for 120 consecutive steps, without colliding. This definition prevents the agent from grazing the goal once and then drifting away.

4.2 Observation and Dynamics Perturbations

All experiments in this paper use the same stochastic perturbation model. The reward combines a goal bonus, dense progress shaping, directional alignment, obstacle repulsion, a clearance bonus, a smoothness penalty, a near-miss penalty, a stall penalty, and a small per-step time cost. The state estimate is deliberately corrupted before it reaches the policy: position is perturbed by Gaussian noise with standard deviation 0.050 m, velocity by 0.100 m/s, the goal vector by 0.050 m, and obstacle positions by 0.080 m.

We then add dynamics and geometry perturbations that make memorized straight-line behavior brittle. The UAV is subjected to aerodynamic drag with coefficient 0.12 and wind gust noise with standard deviation 0.35. Obstacles drift around anchored nominal locations with per-step drift standard deviation 0.018, damping 0.965, and a maximum anchored offset of 0.300 m.

4.3 Kalman Filtering and State Estimation

Under the stochastic perturbation model, the policy no longer sees the exact UAV state. Instead, the environment corrupts position and velocity with Gaussian noise and then fuses them through a Kalman filter:

$$\hat{\mathbf{x}}_{t|t-1} = \mathbf{F}\hat{\mathbf{x}}_{t-1|t-1}, \tag{7}$$

$$\mathbf{P}_{t|t-1} = \mathbf{F}\mathbf{P}_{t-1|t-1}\mathbf{F}^{\top} + \mathbf{Q}, \tag{8}$$

$$\mathbf{K}_t = \mathbf{P}_{t|t-1}\left(\mathbf{P}_{t|t-1} + \mathbf{R}\right)^{-1}, \tag{9}$$

$$\hat{\mathbf{x}}_{t|t} = \hat{\mathbf{x}}_{t|t-1} + \mathbf{K}_t\left(\mathbf{z}_t - \hat{\mathbf{x}}_{t|t-1}\right), \tag{10}$$

where $\hat{\mathbf{x}} = [x, y, v_x, v_y]^{\top}$ and $\mathbf{F}$ is the constant-velocity transition matrix. In our implementation, the filter does not alter the physical state. It only improves the observation delivered to the policy. This distinction matters: the learned agent

Table 2. Training configurations used for the reported policy-comparison, Kalman-ablation, and distillation experiments.

Policy	MLP widths	n_{env}	Learning rate	γ	Batch	Replay / rollout	Update settings
SAC (Kalman)	256-256-256	10	3.0×10^{-4}	0.99	256	replay 300k	start 5k, freq 1, grad 1, $\tau = 0.005$, ent=auto
SAC (no KF)	256-256-256	8	3.0×10^{-4}	0.99	256	replay 300k	start 5k, freq 1, grad 1, $\tau = 0.005$, ent=auto
TD3	256-256-256	8	4.07×10^{-4}	0.99	1024	replay 500k	start 10k, freq 2, grad 2, $\tau = 0.005$, noise 0.10, delay 2
DDPG	256-256	8	1.12×10^{-4}	0.995	1024	replay 800k	start 10k, freq 2, grad 1, $\tau = 0.005$, noise 0.15
PPO	512-256-256	8	2.34×10^{-5}	0.995	1024	rollout 2048 / env	15 epochs, GAE 0.97, clip 0.151, ent 5.06×10^{-5}

still has to act under noisy goals, drifting obstacles, drag, and wind, but it acts from a better state estimate.

The bottom row of Fig. 2 shows why this matters empirically. The Kalman-filtered SAC run learns substantially faster and more stably than the matched no-KF variant, whereas the no-KF run improves more slowly and later collapses in final evaluation. In other words, the Kalman layer does not solve the task by itself; it improves the state estimate enough to make the downstream RL controller less brittle.

4.4 Training Protocol

All policies discussed in the paper are trained directly in the same stochastic setting. SAC is represented by a single-shot run with Kalman filtering enabled. TD3, DDPG, and PPO were tuned with Optuna and then trained on the same noisy environment with 8 parallel environments. These baselines also trained with Kalman-filtered observations; the filter is not unique to SAC. We also trained a matched SAC run without the Kalman filter to isolate the effect of state estimation.

The explainability and distillation sections focus on the SAC controller that yielded the strongest and most stable analytical surrogate under the shared noisy evaluation setting.

Table 2 summarizes the principal hyperparameters of the runs reported in the paper. All controllers use ReLU-activated multilayer perceptrons. For off-policy methods, the listed width is used for both actor and critic streams; for PPO, the same width pattern is used for the policy and value networks. SAC used automatic entropy tuning in the trainer, while TD3 and DDPG used additive Gaussian action noise.

5 Result Analysis

5.1 Algorithm Comparison

As shown in the top row of Fig. 2, SAC reaches reliable success fastest and remains the strongest trace across the comparison window, while TD3 and DDPG

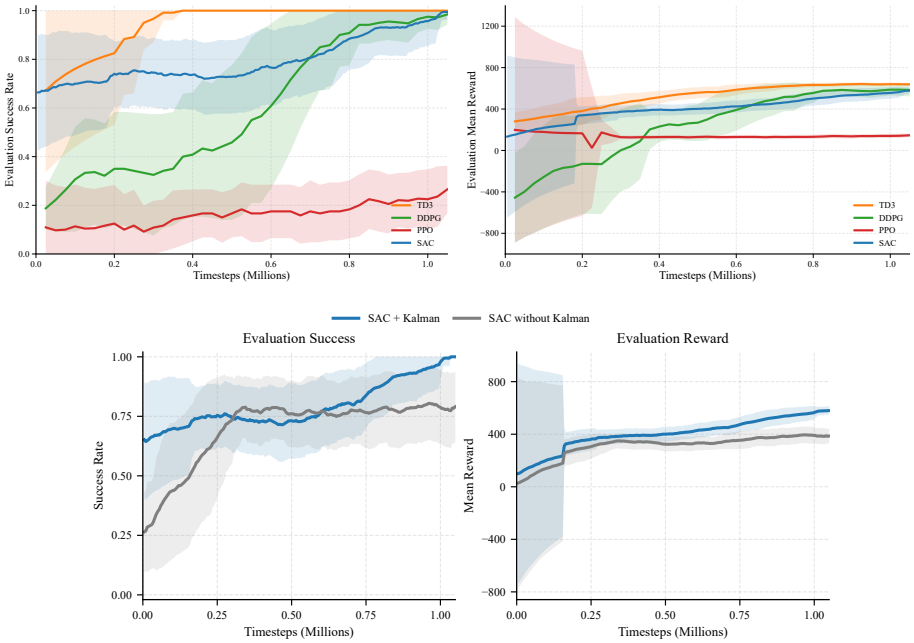

Fig. 2. Top row: evaluation success rate and mean reward for directly trained SAC, TD3, DDPG, and PPO runs in the shared noisy setting, clipped to the first 1.050 M timesteps. Bottom row: matched SAC training with and without Kalman filtering.

both attain perfect checkpoints but exhibit sharper oscillations and later regressions. PPO remains the weakest baseline in this noisy environment, topping out at a best recorded success rate of 62.5%; within these runs, this is consistent with its higher on-policy data demand and with SAC's smoother stochastic control law under the same perturbations.

Within the comparison window, SAC combines fast rise, sustained high success, and the cleanest later behavior, whereas TD3 and DDPG are more consistent with the familiar deterministic off-policy failure mode in which critic drift and reward-shaping loopholes produce strong checkpoints followed by regression.

Table 3 summarizes the strongest checkpoints and final distillation results. The important point is that high task success does not guarantee faithful distillation.

Table 3. Summary of best policy checkpoints and final SHAP-equation fidelity in the shared stochastic setting.

Policy	Best eval succ.	Eq. succ.	Trace dev. (m)
SAC	1.000	1.000	2.154
TD3	1.000	0.000	7.075
DDPG	1.000	0.500	3.557
PPO	0.625	–	–

Note. We did not attempt PPO distillation because the learned PPO neural policy never became strong or stable enough to justify surrogate extraction.

6 Explaining the Learned Policy

6.1 Potential-Field-Like Behavior

Figure 3 provides the clearest qualitative evidence that the main SAC policy learned a control strategy reminiscent of an artificial potential field [10] even though such a field was not explicitly coded into the reward. The free-space action vectors bend around obstacles, the Q-value landscape forms a smooth basin around safe goal-reaching corridors, and the APF comparison makes the behavioral difference explicit: an untuned handcrafted potential field gets trapped in a non-goal local minimum, whereas the learned SAC policy accepts a temporary detour and still reaches the goal.

6.2 Decision Trees, LIME, and SHAP

For the main SAC policy, the saved depth-15 decision tree acts as a near-exact rule-based proxy with $R_x^2 = 0.993$ and $R_y^2 = 0.991$ on evaluation states. The SHAP and LIME panels in Fig. 4 make the same structure explicit: goal displacement dominates attribution, velocity acts as a damping regulator, and obstacle terms appear as more local corrective signals. Together with the near-perfect tree proxy, this stable SHAP ordering argues against the SAC policy being an arbitrary high-frequency map; instead, it behaves like a structured goal-seeking controller with sparse obstacle-triggered corrections.

6.3 Rollout-Aware Equation Distillation

Based on these explanations, we fit a family of compact analytical controllers using raw goal displacement, velocity damping, and short-range obstacle repulsion terms. Importantly, we select the final equation by rollout success first, then trace deviation, and only then by one-step R^2. This avoids the common failure mode where a surrogate looks excellent on sampled states but diverges quickly in closed loop.

Table 4 shows why the final SAC law is not selected by one-step fit alone. The richer compact and contextual bases achieve much higher R^2_{full}, but both fail in closed loop. The lower-order legacy compact form is therefore retained because it is the only analytical family that preserves all evaluation successes.

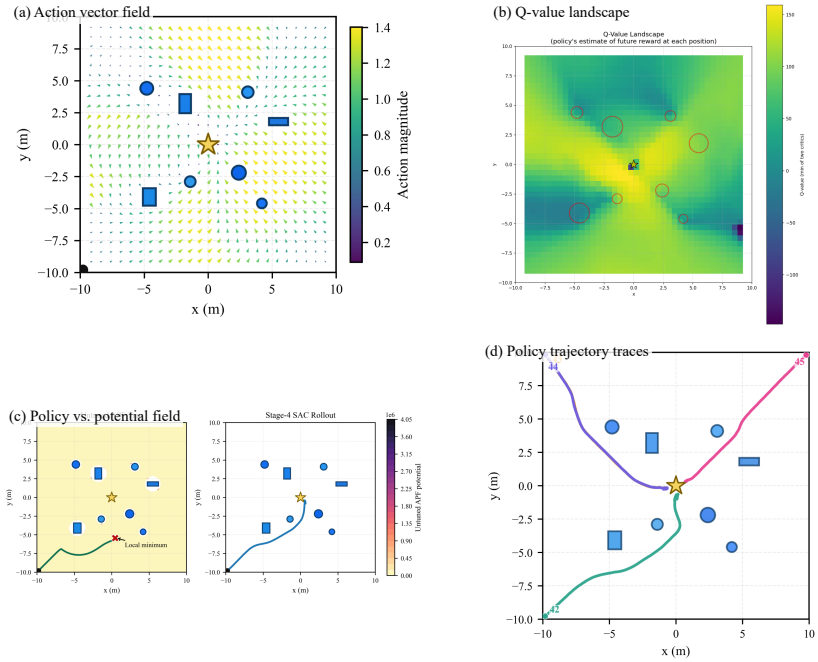

Fig. 3. Interpretability panel for the main SAC policy: action vectors, Q-value landscape, APF failure case, and representative trajectories.

Table 4. Numerical comparison of SAC equation families considered during rollout-aware distillation. Reported rollout metrics are measured on the fixed evaluation seeds used to choose the final analytical controller.

Equation family	Best R^2_{full}	Success	Trace dev. (m)	Endpoint dev. (m)
Legacy nominal compact	0.624	4/4	2.808	0.799
Compact basis	0.814	0/4	7.210	6.460
Contextual basis	0.814	0/4	7.289	6.723

Let $(e_x, e_y) = (g_x - x, g_y - y)$ denote goal-relative displacement, let (v_x, v_y) denote the Kalman-filtered planar velocity estimate, and let (r_x, r_y) denote the nearest-obstacle repulsion vector with exponential decay inside a 4.500 m threshold.

Final distilled SAC law.

$$a_x = 0.0427 + 0.1442\, e_x - 0.0120\, v_x + 0.4725\, r_x, \tag{11}$$

$$a_y = -0.0624 + 0.1144\, e_y - 0.0071\, v_y + 0.6592\, r_y. \tag{12}$$

In plain language, Eq. (11) says: pull the UAV toward the goal, subtract a fraction of the current velocity to damp overshoot, and add a short-range obstacle-normal correction when nearby clearance collapses. In control terms, it is closest to a damped artificial-potential-field law: proportional attraction toward the

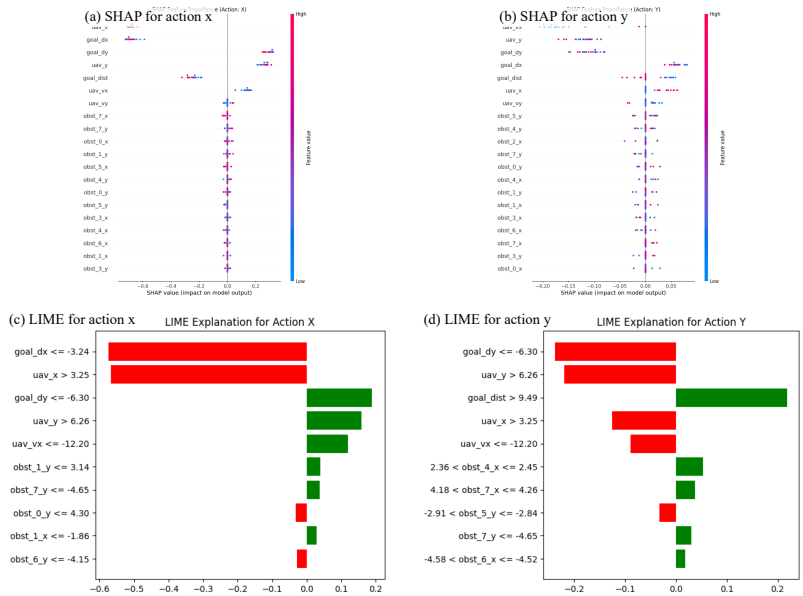

Fig. 4. Post hoc explanations for the main SAC policy using SHAP and LIME. Goal displacement dominates globally, while obstacle-aware corrections remain locally important.

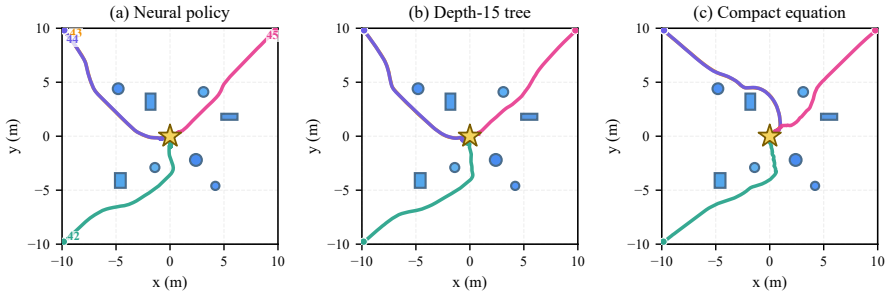

Fig. 5. Held-out SAC trajectories for the executable surrogates emphasized in the paper: the neural network policy, the depth-15 decision tree, and the final compact SHAP-guided equation. The tree is nearly indistinguishable from the policy, while the compact equation preserves the same successes with slightly larger detours.

goal, viscous damping in velocity, and a local repulsive correction around obstacles. The importance of this form is not just that it matches the learned policy numerically, but that each term maps onto a familiar physical control primitive that can be inspected and manually retuned. On evaluation rollouts, this equation preserved 4/4 successes and achieved a mean trajectory deviation of 2.154 m.

Figure 5 highlights the main positive result more directly than a cross-algorithm failure comparison. For the SAC controller, both the large decision

tree and the compact equation remain executable surrogates on the evaluation seeds. The tree is effectively a near-exact replacement, while the equation preserves the same qualitative behavior with a modest increase in path curvature and travel time. Other algorithms did not yield equally faithful compact equations; those weaker surrogate results are summarized numerically in Table 3 and revisited briefly in the discussion rather than being made the visual centerpiece of the paper.

7 Discussion and Limitations

The most important lesson from these experiments is that policy quality and policy distillability are related but not identical. In the shared noisy setting, SAC, TD3, and DDPG can all reach perfect neural-policy checkpoints, yet only SAC consistently yields a compact analytical surrogate with acceptable closed-loop fidelity. Since SAC and TD3 share the same 256-256-256 actor depth in our implementation, this gap is unlikely to be a simple consequence of network size alone; the more relevant difference is the combination of stochastic policy head, twin-critic structure, and regularized training objective shown in Fig. 1.

The scope of the claim is intentionally narrow. We are not proposing a new RL algorithm, nor proving that entropy alone causes distillability. Rather, we show that in a custom but carefully instrumented UAV-navigation benchmark, one successful controller family can be recovered as a compact physical law while others remain harder to compress into the same analytical family. The environment is still planar in motion and the explanation pipeline remains post hoc, so broader claims will require additional tasks, stronger causal ablations, and eventually real-flight validation.

8 Conclusion

We introduced a custom Gymnasium/PyBullet UAV-navigation environment with planar motion embedded in a 3D simulator and evaluated all reported policies in a single-shot noisy setting that combines corrupted observations, Kalman-filtered state estimation, drag, wind, and drifting obstacles. We then analyzed the learned policies with decision trees, LIME, SHAP, vector fields, Q-value heatmaps, and trajectory-level distillation.

The central finding is that SAC is the most *distillable* policy among the controllers tested here and also the most stable performer in the saved noisy runs. Its final SHAP-guided equation preserves rollout success and exposes a compact damped potential-field-like controller that was never hard-coded. This suggests a useful research direction for future work: evaluating learned policies not only by reward and success rate, but also by the fidelity of the simplest analytical controller that can reproduce them.

References

1. Bastani, O., Pu, Y., Solar-Lezama, A.: Verifiable reinforcement learning via policy extraction. In: Advances in Neural Information Processing Systems 31. pp. 2494–2504 (2018)
2. Bekkemoen, Y., Langseth, H.: Interpretable deep reinforcement learning via concept-based policy distillation. Machine Learning **114**, 288 (2025). https://doi.org/10.1007/s10994-025-06928-5
3. Brockman, G., Cheung, V., Pettersson, L., Schneider, J., Schulman, J., Tang, J., Zaremba, W.: Openai gym (2016). https://doi.org/10.48550/arXiv.1606.01540
4. Coumans, E., Bai, Y.: Pybullet, a python module for physics simulation for games, robotics and machine learning. http://pybullet.org (2021), software
5. Fujimoto, S., van Hoof, H., Meger, D.: Addressing function approximation error in actor-critic methods. In: Proceedings of the 35th International Conference on Machine Learning. PMLR, vol. 80, pp. 1587–1596 (2018)
6. Haarnoja, T., Zhou, A., Abbeel, P., Levine, S.: Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In: Proceedings of the 35th International Conference on Machine Learning. PMLR, vol. 80, pp. 1861–1870 (2018)
7. Huang, Z., Yang, Z., Krupani, R., Şenbaşlar, B., Batra, S., Sukhatme, G.S.: Collision avoidance and navigation for a quadrotor swarm using end-to-end deep reinforcement learning (2024). https://doi.org/10.48550/arXiv.2309.13285, accepted to ICRA 2024
8. Kalman, R.E.: A new approach to linear filtering and prediction problems. Journal of Basic Engineering **82**(1), 35–45 (1960). https://doi.org/10.1115/1.3662552
9. Kästner, L., Bhuiyan, T., Le, T.A., Treis, E., Cox, J., Meinardus, B., Kmiecik, J., Carstens, R., Pichel, D., Fatloun, B., Khorsandi, N., Lambrecht, J.: Arena-bench: A benchmarking suite for obstacle avoidance approaches in highly dynamic environments (2022). https://doi.org/10.48550/arXiv.2206.05728
10. Khatib, O.: Real-time obstacle avoidance for manipulators and mobile robots. The International Journal of Robotics Research **5**(1), 90–98 (1986). https://doi.org/10.1177/027836498600500106
11. Lee, K., Kim, S., Choi, J.: Adaptive and explainable deployment of navigation skills via hierarchical deep reinforcement learning (2023). https://doi.org/10.48550/arXiv.2305.19746
12. Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., Wierstra, D.: Continuous control with deep reinforcement learning (2015). https://doi.org/10.48550/arXiv.1509.02971
13. Lundberg, S.M., Lee, S.I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems 30. pp. 4765–4774 (2017)
14. Marino, F., Guglieri, G.: Beyond static obstacles: Integrating kalman filter with reinforcement learning for drone navigation. Aerospace **11**(5), 395 (2024). https://doi.org/10.3390/aerospace11050395
15. Milani, S., Topin, N., Veloso, M., Fang, F.: A survey of explainable reinforcement learning. arXiv preprint arXiv:2202.08434 (2022). https://doi.org/10.48550/arXiv.2202.08434
16. Peng, X.B., Andrychowicz, M., Zaremba, W., Abbeel, P.: Sim-to-real transfer of robotic control with dynamics randomization. In: 2018 IEEE International Conference on Robotics and Automation. pp. 3803–3810 (2018). https://doi.org/10.1109/ICRA.2018.8460528

17. Ribeiro, M.T., Singh, S., Guestrin, C.: "why should i trust you?": Explaining the predictions of any classifier. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. pp. 1135–1144 (2016). https://doi.org/10.1145/2939672.2939778
18. Rusu, A.A., Colmenarejo, S.G., Gulcehre, C., Desjardins, G., Kirkpatrick, J., Pascanu, R., Mnih, V., Kavukcuoglu, K., Hadsell, R.: Policy distillation (2015). https://doi.org/10.48550/arXiv.1511.06295
19. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., Klimov, O.: Proximal policy optimization algorithms (2017). https://doi.org/10.48550/arXiv.1707.06347
20. Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., Abbeel, P.: Domain randomization for transferring deep neural networks from simulation to the real world. In: 2017 IEEE/RSJ International Conference on Intelligent Robots and Systems. pp. 23–30 (2017). https://doi.org/10.1109/IROS.2017.8202133